Build a Production ML Pipeline with MLflow, DVC, and Automated Retraining

Back

Build

Domain

Data Engineering

Data Science

Roles

Data Scientist

DevOps Engineer

Machine Learning Engineer

Skills

API Development | Artificial Intelligence | Automated Testing | Bug Tracking | Data Cleaning | Model Versioning | Monitoring | System Design

Difficulty

Hard

Tools

Docker | Evidently AI | FastAPI | Great Expectations | HG | MLflow | PostgreSQL | Pytest | Python

Industries

E-commerce | Finance

Pending Submission

Please submit your work when ready.

TIME REMAINING

4d

Deadline: **17 Oct 2026 at 04:59 PM**

Overview | Instructions | Resources | Submit

Objective

You will build a complete, end-to-end MLOps pipeline for a machine learning model. You will learn to implement automated training, data versioning with DVC,

experiment tracking with MLflow, containerized API deployment, production monitoring for data drift, and a feedback loop for automated retraining. This project mirrors how production ML systems are built and maintained, providing critical skills for deploying robust and reliable AI products.

Description

Background

In modern data science, building a predictive model is only the first step. The real challenge lies in deploying, monitoring, and maintaining that model in a production environment. This discipline, known as MLOps (Machine Learning Operations), combines ML, DevOps, and data engineering to manage the complete ML lifecycle. Production models often suffer from performance degradation over time due to phenomena like *data drift* (the statistical properties of input data change) and *concept drift* (the relationship between input and output variables changes). A robust MLOps pipeline is essential to detect and mitigate these issues automatically.

This project will guide you through building such a pipeline. You will use industry-standard tools to manage each stage:

1. **DVC (Data Version Control):** To version control datasets and intermediate data files, ensuring reproducibility, much like Git does for code.
2. **MLflow:** To track experiments (parameters, metrics, artifacts), package models for deployment, and manage the model lifecycle with a central Model Registry.
3. **FastAPI:** To serve the trained model as a high-performance, containerized REST API.
4. **Docker:** To package the application, its dependencies, and services like MLflow into portable containers, ensuring consistency across environments.
5. **Drift Detection:** To monitor the live API traffic and detect statistical drift between the training data and production data, which serves as a trigger for retraining.

By completing this task, you will create a system that not only serves predictions but also intelligently maintains its own performance over time.

Implementation Details

Follow these steps to build the complete MLOps pipeline. You are free to choose a standard classification or regression dataset (e.g., customer churn, credit fraud,

income prediction).

Step 1: Project Setup and Data Ingestion

1. Create a project directory with a structure that separates concerns. We recommend the following structure:

```
.
├── data/
│   ├── raw/          # Raw, immutable data
│   └── processed/    # Processed data for training
├── notebooks/        # Jupyter notebooks for exploration
├── reports/          # For drift detection reports
├── src/              # Source code for the pipeline
│   ├── __init__.py
│   ├── api/          # FastAPI application
│   ├── config.py     # Configuration management
│   ├── data_processing.py
│   ├── monitoring.py
│   └── train.py
├── tests/            # Unit and integration tests
├── .env.example
├── docker-compose.yml
├── Dockerfile
├── dvc.yaml
└── README.md
```

2. Select a dataset and place it in `data/raw/`. Initialize DVC to start tracking this data as specified in requirement `dvc-data-versioning`.

Step 2: Data Validation and Processing

1. Implement a data processing script (`src/data_processing.py`) that cleans the raw data and performs feature engineering. The output should be saved to `data/processed/`.
2. Integrate a data validation tool like Great Expectations to define expectations for your data. This should be a distinct step in your pipeline, as per requirement `great-expectations-validation`.

Step 3: Model Training and Experiment Tracking

1. Develop your model training script (`src/train.py`). This script should load processed data, train a model, and evaluate it.
2. Integrate MLflow to log experiment runs. Each run should log parameters, metrics (e.g., accuracy, F1-score), and the trained model artifact itself. See requirement `mlflow-experiment-logging`.

3. After experimenting, register the best performing model version in the MLflow Model Registry, as required by `mlflow-model-registry`.

Step 4: Model Serving with a REST API

1. Create a FastAPI application in `src/api/` that loads the production-stage model from the MLflow Model Registry.
2. Implement a health check endpoint (`/health`) and a prediction endpoint (`/predict`), fulfilling requirements `api-health-check` and `api-prediction-endpoint`.
3. The prediction endpoint should accept a JSON payload with features and return the model's prediction.

Step 5: Containerization

1. Write a `Dockerfile` for your FastAPI application.
2. Create a `docker-compose.yml` file to orchestrate all services: your API, an MLflow tracking server, and a PostgreSQL database for MLflow's backend. Ensure all services have health checks as specified in `docker-compose-setup`.

```
# docker-compose.yml template
version: '3.8'

services:
  db:
    image: postgres:13
    container_name: mlflow_db
    environment:
      - POSTGRES_USER=${POSTGRES_USER}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
      - POSTGRES_DB=${POSTGRES_DB}
    ports:
      - "5432:5432"
    volumes:
      - ./postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES
      interval: 10s
      timeout: 5s
      retries: 5

  mlflow-server:
    image: ghcr.io/mlflow/mlflow:v2.11.1 # Use an official image
    container_name: mlflow_server
    depends_on:
      db:
```

```
        condition: service_healthy
    ports:
      - "5000:5000"
    environment:
      - MLFLOW_BACKEND_STORE_URI=postgresql://${POSTGRES_USER}:${POSTGI
      - MLFLOW_ARTIFACT_ROOT=/mlruns
    volumes:
      - ./mlruns:/mlruns
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000"]
      interval: 30s
      timeout: 10s
      retries: 3

  api:
    build: .
    container_name: ml_api
    depends_on:
      mlflow-server:
        condition: service_healthy
    ports:
      - "8000:8000"
    environment:
      - MLFLOW_TRACKING_URI=http://mlflow-server:5000
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 30s
```

Step 6: Monitoring and Automated Retraining

1. Implement a script (`src/monitoring.py`) that simulates monitoring. This script should compare a batch of recent prediction inputs against the training data to detect drift. Use a library like Evidently AI.
2. The script should generate a drift report, satisfying `drift-detection-report` .
3. Create a trigger mechanism for retraining (`retraining-trigger-script`). If significant drift is detected, this mechanism should execute the training pipeline to produce a new model candidate, fulfilling `retraining-logic` .

Step 7: Submission Artifacts

Ensure your final submission is a complete Git repository containing all of the following artifacts.

- [ ] `README.md` : A detailed document explaining your project, architecture, and instructions to run it.

- [ ] `docker-compose.yml` : The file to orchestrate all services.
- [ ] `Dockerfile` : For building the API service container.
- [ ] `.env.example` : Template for all required environment variables.
- [ ] `src/` : All Python source code for data processing, training, API, and monitoring.
- [ ] `tests/` : Directory with unit and integration tests.
- [ ] `dvc.yaml` and `.dvc/config` : DVC pipeline and configuration files.
- [ ] `requirements.txt` : A list of all Python dependencies.
- [ ] All other necessary configuration files.

Implementation Guidelines

1. **Pipeline Orchestration:** While a full orchestrator like Airflow is out of scope, consider how you structure your DVC pipeline (`dvc.yaml`) or shell scripts to represent dependencies between data processing, validation, and training stages.
2. **Model Promotion Logic:** Think about the criteria for promoting a newly retrained model to production. It should not only be triggered by drift but also outperform the current champion model on a held-out test set.
3. **API Security:** For a real-world application, you would add authentication and authorization to your API endpoints. While not a core requirement, consider where you would add this layer (e.g., using FastAPI's dependency injection system for an API key header).
4. **Configuration Management:** Avoid hardcoding values like file paths, model names, or parameters. Use a configuration file (e.g., YAML) or environment variables, managed by a library like Pydantic, to make your pipeline configurable.
5. **Testing:** Write unit tests for your data processing logic and helper functions. Write integration tests for your API endpoints to ensure they behave as expected.

FAQ

1. **Q: Which dataset should I use?**
    1. A: You can use any public classification or regression dataset. The UCI ML Repository or Kaggle are good sources. For simulating drift, the "Credit Card Fraud Detection" dataset from Kaggle is a good choice, as you can simulate drift by training on one time period and monitoring another.
2. **Q: How can I simulate data drift for testing the monitoring and retraining loop?**

    1. A: You can manually alter a batch of production data before sending it to the monitoring service. For example, you could scale a numerical feature by a constant factor or change the distribution of a categorical feature.
3. **Q: Do I need a cloud account or to deploy to Kubernetes?**
    1. A: No. The entire system must be runnable locally using Docker Compose. All services should be containerized and orchestrated by the `docker-compose.yml` file.
4. **Q: What if my model performance is not state-of-the-art?**
    1. A: The primary goal is to build the MLOps pipeline, not to achieve the highest possible model score. Focus on the infrastructure, automation, and processes. A reasonably performing model is sufficient.
5. **Q: The `docker-compose.yml` specifies a specific MLflow image. Can I use another?**
    1. A: Yes, you can use any recent, stable version of the official MLflow image. The key is that the service runs correctly and integrates with the database and your API, as per requirement `mlflow-tracking-server`.

Core Requirements

1. The entire application stack must be defined and orchestrated using a `docker-compose.yml` file. This file should define, at a minimum, an application service (`api`), an MLflow service (`mlflow-server`), and a database service (`db`) for the MLflow backend. All services must include a `healthcheck` to ensure they start in the correct order and are running properly.

   File Location: `docker-compose.yml` (at the repository root)

   Required Services:

   1. `db` : A PostgreSQL service for the MLflow backend.
   2. `mlflow-server` : The MLflow tracking server.
   3. `api` : Your FastAPI application for model serving.

   Healthcheck Requirements:

   1. `db` : Must use `pg_isready` .
   2. `mlflow-server` : Must check if the MLflow UI is accessible (e.g., via `curl`).
   3. `api` : Must check the `/health` endpoint.

   Verification:

   1. Run `docker-compose up --build` .
   2. Verify that all containers start without errors and become healthy within 5 minutes.
   3. Verify that the `api` and `mlflow-server` services correctly wait for the `db` service to be healthy before starting.

2. A `.env.example` file must be present in the root of the repository, documenting all environment variables required to run the application. This includes credentials for the database, URIs for services, and any other configuration variables.

   File Location: `.env.example` (at the repository root)

   Required Variables:

   1. `POSTGRES_USER`
   2. `POSTGRES_PASSWORD`
   3. `POSTGRES_DB`
   4. `MLFLOW_TRACKING_URI`
   5. Any other variables used by your application.

   Verification:

   1. Check for the existence of the `.env.example` file.
   2. Verify that it contains all the necessary variables listed above with placeholder values.
   3. Verify that no real secrets are committed.

---

3. The raw dataset must be versioned using DVC. A `dvc.yaml` file should define at least one pipeline stage for data processing.

   Required Files:

   1. `data/raw/<dataset-file>.dvc`: A `.dvc` file pointing to your raw dataset.
   2. `dvc.yaml`: A DVC pipeline definition file.

   Verification:

   1. Check for the existence of `data/raw/<dataset-file>.dvc`.
   2. Check for the existence of `dvc.yaml`.
   3. The `dvc.yaml` should define at least one stage (e.g., `process_data`) with dependencies, parameters, and outputs.
   4. Run `dvc status` to ensure the pipeline state is consistent.

---

4. A data validation step using a library like Great Expectations must be part of your pipeline. This step should generate a validation report for the training data.

   Expected Output:

   1. A validation results file must be generated by the process.
   2. For Great Expectations, this would be a JSON file located in a path similar to `great_expectations/uncommitted/validations/your_suite/results.json`.

   Verification:

   1. Execute the data pipeline.
   2. Verify that a validation results file is created.
   3. Parse the JSON file and confirm the `success` key is present and is a boolean value.

---

5. The MLflow tracking server must be accessible via its UI. The server must be configured to use the PostgreSQL database as a backend store.

   Service: `mlflow-server` (in `docker-compose.yml`)

   Port: The service should expose port 5000.

   Verification:

   1. After `docker-compose up`, send an HTTP GET request to `http://localhost:5000`.
   2. Verify the response status code is 200 OK.
   3. Verify the response body contains HTML with the title "MLflow".

---

6. The training script must connect to the MLflow tracking server and log experiments. At least one experiment must be created, containing a minimum of three distinct runs.

   MLflow Tracking URI: `http://mlflow-server:5000`

   Requirements:

   1. An experiment with a specific name (e.g., `churn-prediction`) must be created.
   2. This experiment must contain at least 3 runs.
   3. Each run must log at least one parameter (e.g., `learning_rate`) and one metric (e.g., `f1_score`).

   Verification:

   1. Use the MLflow client API to connect to `http://localhost:5000`.
   2. Fetch the list of experiments and verify the named experiment exists.
   3. List the runs for that experiment and verify the count is >= 3.
   4. Inspect each run to confirm that parameters and metrics are logged.

---

7. The best performing model from your experiments must be registered in the MLflow Model Registry under a specific name. This model should have at least one version, and that version should be transitioned to the 'Production' stage.

   Model Name: `production-model`

   Required Stage: `Production`

   Verification:

   1. Use the MLflow client API to connect to `http://localhost:5000`.
   2. Search for the registered model named `production-model`.
   3. Verify that the model exists.
   4. Fetch the latest versions of the model and confirm at least one version is in the 'Production' stage.

---

8. The FastAPI application must expose a `/health` endpoint for monitoring and health checks.

   Endpoint: `GET /health`

   Response (200 OK):

   ```
   {
     "status": "ok"
   }
   ```

   Verification:

   1. Send an HTTP GET request to `http://localhost:8000/health`.
   2. Verify the response status code is 200.
   3. Verify the response body is a JSON object matching the specified schema.

---

9. The API must provide a `/predict` endpoint that accepts a JSON payload of features and returns a model prediction. The API should load the 'Production' stage model from the MLflow registry to make predictions.

   Endpoint: `POST /predict`

   Request Schema (Example):

   ```
   {
     "feature1": 0.5,
     "feature2": 1.2,
     "category_feature": "A"
   }
   ```

   Response Schema (200 OK):

   ```
   {
     "prediction": 0
   }
   ```

   Verification:

   1. Send a POST request to `http://localhost:8000/predict` with a valid JSON payload matching the model's expected features.
   2. Verify the response status code is 200.
   3. Verify the response body matches the specified schema and contains a `prediction` key.

---

10. The `/predict` endpoint must gracefully handle invalid or incomplete input by returning a 422 Unprocessable Entity status code, as is standard for FastAPI.

    Endpoint: `POST /predict`

    Invalid Request (Example):

    ```
    {
      "feature1": "this-should-be-a-float"
    }
    ```

    Response:

    1. **Status Code:** 422 Unprocessable Entity

    Verification:

    1. Send a POST request to `http://localhost:8000/predict` with a malformed JSON payload (e.g., incorrect data types, missing fields).
    2. Verify the response status code is 422.

---

11. A script must be provided to perform data drift detection between a reference dataset (e.g., training data) and a current dataset (e.g., recent production data). This script must generate an HTML report.

    Script Location: A runnable script (e.g., `src/monitoring.py` or a shell script).

    Output File: `reports/data_drift_report.html`

    Verification:

    1. Execute the drift detection script.
    2. Verify that the file `reports/data_drift_report.html` is created.
    3. Verify the file is a valid HTML document and is not empty.

---

12. A runnable script must be provided to trigger the model retraining pipeline. This script should execute the training process and register a new model version in MLflow.

    Script Location: e.g., `retrain.sh` or `python -m src.retrain`

    Verification:

    1. Check the current number of model versions for `production-model` in the MLflow registry.
    2. Execute the retraining script.
    3. Check the MLflow registry again and verify that a new model version has been created.

---

13. The system must include logic that conditionally triggers retraining based on detected drift. This can be simulated by checking for the existence of a flag file.

    Trigger Condition: The retraining process should only run if a flag file named `drift_detected.flag` exists in the repository root.

    Verification:

    1. Ensure the `drift_detected.flag` file does not exist. Run the retraining trigger script and verify that no new model is trained.
    2. Create an empty file named `drift_detected.flag`.
    3. Run the retraining trigger script again.
    4. Verify that a new MLflow run is created and a new model version is registered in MLflow.

---

14. The project must include a suite of unit tests for critical components, such as data processing functions. The test suite must achieve a minimum of 60% code coverage.

    Test Location: `tests/` directory.

    Execution: Tests should be runnable with a command like `pytest`.

    Coverage Requirement: >= 60%

    Verification:

    1. Run the test suite with coverage enabled (e.g., `pytest --cov=src`).
    2. Parse the coverage report.
    3. Verify that the total coverage is 60% or higher.

---

partnr

About Us Contact Us Privacy Policy

Terms and Conditions

All rights reserved. Copyright, Partnr 2025-26